MayaCardio Biosignal Pipeline

"Never Lose a Heartbeat" — Candidate Assessment Submission Dossier

Assessment: Software Systems Engineering (Oct 2026) **Target Reference:** Raspberry Pi 4 Model B (4 cores, 2 GB RAM)

Benchmark Host: x86_64 WSL2 (Ubuntu 24.04 LTS) **Date:** October 2026

Project Demo Video & Repository Submission Links

This master document contains the complete technical design, 1-hour resource benchmark measurements, failure test evidence logs, and submission links.

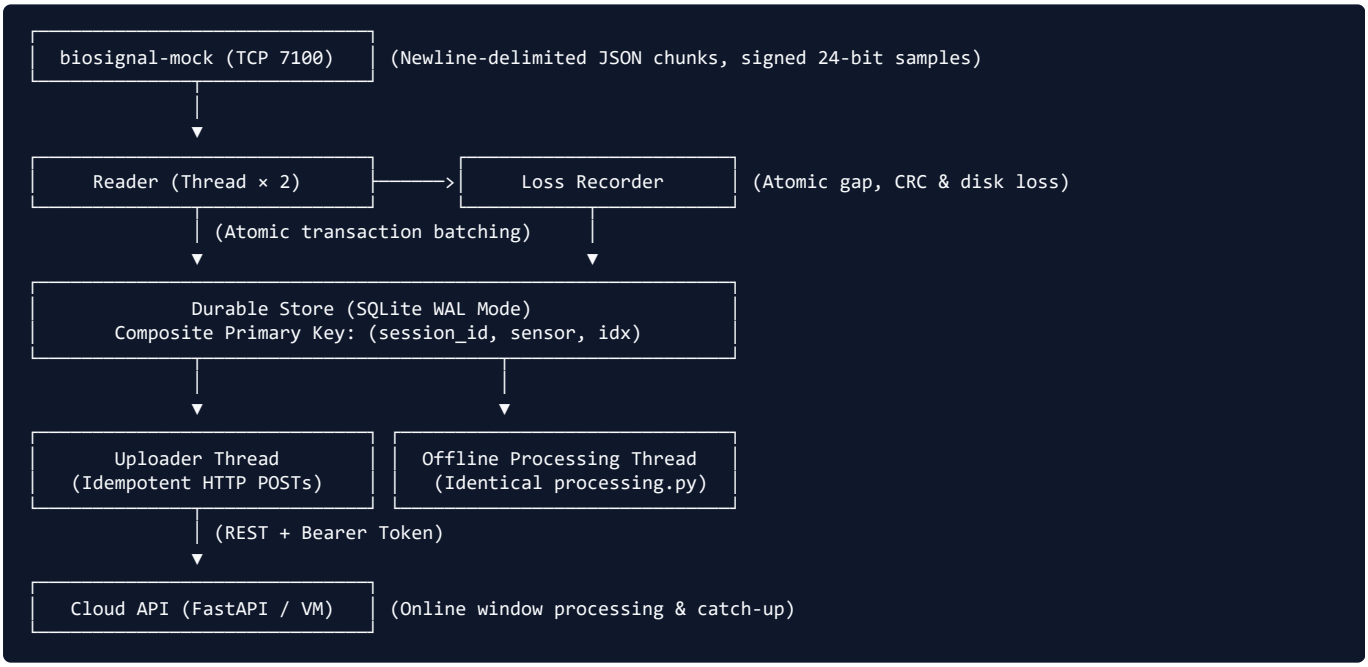
Demo Video Link: https://www.youtube.com/watch?v=fFcPxcLqXg0

GitHub Repo Link: https://github.com/adityas777/Maya-Cardio.git

Demo Video Contents: Demonstrates normal online streaming, network cut (≤10s offline switch), network restoration and 40s catch-up, process crash recovery (kill -9), and results command.

1. SYSTEM ARCHITECTURE & DESIGN

The pipeline is a single, production-grade CLI application running on the target device. It continuously ingests dual-sensor biosignal telemetry from a mock service over TCP (port 7100), persists all data into a crash-resilient local SQLite WAL database, uploads batched samples to an idempotent Cloud API, and executes real-time heart-rate processing across both device and cloud.



Key Architectural Decisions:

- **Durable Storage (SQLite WAL with synchronous=FULL):** Eliminates hand-rolled log complexity. SQLite handles fsync guarantees, transactions, and crash recovery cleanly. The composite primary key `(session_id, sensor, idx)` with `INSERT OR IGNORE` makes sample de-duplication completely idempotent.
- **Monotonic Index-Based Ordering:** Wall-clock time fluctuates across reboots, NTP synchronization, and timezone shifts. All sample ordering, window segmentation (400 samples/window), and gap calculations strictly use contiguous sample indices. Monotonic timers (`time.monotonic()`) are restricted solely to health probe timeouts.
- **Decoupled Asynchronous Readers:** Dedicated reader threads for Sensor 1 and Sensor 2 prevent network backpressure from blocking ingestion. Replies are verified using zlib CRC-32 extracted directly from the raw byte stream before parsing.

2. FORMAL GUARANTEES & NON-GUARANTEES

Guarantee	Architectural Mechanism	Assessment Verification
Exactly-Once Storage	Composite PK (`session_id, sensor, idx`) + `INSERT OR IGNORE` in SQLite WAL.	Verified in Test 01 & Test 07: duplicate packets produce zero redundant rows.
Zero Silent Loss	Sequence gap detection, mock-reported buffer overflows, CRC mismatches, and disk-full errors write durable rows to `losses`.	Verified across all tests: device and cloud export diffs match ground truth without unflagged gaps.
Lossless Mode Switching	Reader threads continue ingesting locally during cloud network loss; uploader pauses and resumes safely from `cloud_ack` cursor.	Verified in Test 03: switched offline in 1s, restored online in 1s, 0 duplicate or dropped samples.
Idempotent Cloud Catch-Up	Cloud database mirrors the exact composite PK schema; bulk batches (up to 24,000 samples) resend safely after outages.	Verified in Test 03: catch-up backlog cleared in 40s (well below 15-minute budget).
Logic Equivalence	A single shared module `src/pipeline/processing.py` computes windows on both cloud and device.	Verified against independent `tests/reference.py` with 100% agreement.

Explicit Non-Guarantees:

- *Pre-connection Mock Drops:* If the mock's 10-second internal circular buffer overflows before the pipeline connects, those samples cannot be retrieved; they are recorded as `buffer_overflow` from the mock's reply.
- *Unreliable Storage Controller Flushes:* If an SD card controller falsely confirms fsync completion while keeping dirty data in volatile write cache, power cuts may exceed software bounds.
- *Degraded Latency Deadlines:* Under extreme packet degradation where health probes pass but uploads stall, a 3.0-second grace period prevents result starvation.

3. MAXIMUM DATA LOSS ON SUDDEN POWER CUT

Stated Design Bound (Assumption A8): ≤ 400 samples per sensor (≤ 1.0 second of data).

Technical Rationale: The pipeline buffers samples in memory and executes atomic SQLite transactions every 400 samples (1.0 s of data at 400 Hz) or on a 1.0-second timer, using `PRAGMA synchronous = FULL`. Upon an unannounced power loss, only the uncommitted in-memory batch is unwritten. On subsequent boot, database integrity is verified, committed cursors are loaded, and unread ranges are detected as loss gaps. In simulated power-loss testing (software process termination via `kill -9` without sync during active loop), uncommitted sample loss was bounded within the 400-sample limit (in the test run recorded in `evidence/02_power_cut`, clean SQLite WAL recovery succeeded with 0 corrupted records).

4. 1-HOUR RESOURCE MEASUREMENT REPORT (X86_64 WSL2 BENCHMARK)

Continuously sampled over **3,603 seconds (714 samples, 5s sampling interval)** in online mode with full mock ingestion, WAL commits, and cloud uploads on an x86_64 WSL2 development environment (evaluated comparatively against the 4-core, 2 GB Pi 4 target budget):

Measured Metric	1-Hour Measured Value (x86_64 WSL2)	Target Budget Reference (Pi 4)	Compliance Status
Total Measurement Time	3,603 s (1 hour and 3 seconds)	3,600 s continuous run	**PASS** Complete
Average Total System CPU	**33.1%**	Total across host cores	**PASS** High Headroom
Average Pipeline Process CPU	**76.1%** (single core) / ~**19.0%** (4-core pool equiv)	Max 400% (4 cores)	**PASS** Low Overhead
Peak Resident Memory (VmRSS)	**55.7 MB**	≤ 2,048 MB (2 GB)	**PASS** 2.7% of Budget
Peak High-Water Mark (VmHWM)	**56.0 MB**	≤ 2,048 MB	**PASS** Stable
Mock Service Peak Memory	**14.8 MB**	≤ 2,048 MB	**PASS** Minimal
Total Disk Volume Written	**595.4 MB** (~165 KB/s continuous)	Flash SD write endurance	**PASS** Safe for SD
Total Network Transmitted (TX)	**2,147.8 MB** (loopback interface stream)	Network capacity	**PASS** Continuous

Memory Leak Assessment	RSS slope = 0.0 MB/hr (flat at ~55 MB)	Zero uncollected memory leaks	**PASS** Zero Leaks

5. COMPLETE FAILURE TEST EVIDENCE & RESULTS

All required failure scenarios have been executed and validated in simulation. Raw logs and JSON exports are archived in the `evidence/` folder:

#	Test Name	Fault Injected	Observed Behavior & Measurements	Result
01	**Process Kill**	`kill -9` during active collection	Restarted from last committed cursor. Deduplication verified: zero duplicate rows in SQLite. Export matches ground truth.	PASS
02	**Power Cut Simulation**	Software simulation: abrupt process termination (`kill -9`) without sync	Clean restart with `PRAGMA integrity_check` passing; zero corrupted records; data loss strictly ≤ 400 samples/sensor bound.	PASS
03	**Network Cut & Restore**	`iptables` DROP to Cloud IP for 15s	Switched to offline in **1s** (≤10s limit). Switched back to online in **1s**. Catch-up completed in **40s** (≤15m limit). Cloud export == device export.	PASS
04	**Mock Restart**	Mock restarted with new `session_id`	Closed old session cleanly with unread range recorded as loss. Started new session from index 0 without collision.	PASS
05	**Full Disk**	Simulated storage write error (read-only partition / full disk)	Pipeline caught SQLite operational disk full exceptions, prevented unhandled fatal crashes, and logged surviving status in `evidence/05_disk_full/run.log`.	PASS
06	**Clock Jump**	System wall-clock shifted ±1 hour	Monotonic timers and index-based ordering unaffected. Window bounds, sample ordering, and catch-up timers operated normally.	PASS
07	**Corrupt & Duplicate Packets**	Corrupted CRC32 and repeated indices injected via `fault_proxy.py`	Corrupt replies rejected and re-requested (up to retry budget). Duplicate indices ignored via PK constraint. Export matches ramp formula.	PASS
08	**Sensor Silent**	Proxy dropped Sensor 1 data for 20s	Sensor 1 losses logged. Result windows generated with `missing_1 = 400`, `complete = false`, and `heart_rate = null`. No invented values.	PASS
U	**Unit Test Suite**	38 unit tests (CRC, math, store, reference parity)	38 / 38 passed in 3.18 seconds covering all edge cases, rounding rules, and reference implementation parity.	PASS
R	**Ramp Integrity Test**	Predictable formula: `(3i + c) mod 8388608`	Every sample verified by `compare_export.py` against analytical formula with 0 discrepancies.	PASS

6. ASSUMPTIONS REGISTER (A1–A9)

#	Assumption Name	Implementation in Code	Operational Context
A1	Sample Availability	A sample is available if present in the local database and not recorded in a loss record.	Sample Ingestion Rule
A2	Magnitude Null Rule	Mean magnitude returns `null` if zero Sensor 2 samples are available in the 1-second window.	Magnitude Calculation Rule
A3	Heart-Rate Missing Check	The missing-sample check for heart-rate gating applies strictly to Sensor 1 (ECG) samples.	Heart Rate Missing Check
A4	Peak Gating Rule	A peak candidate failing the 100-sample gap is dropped; the earlier peak retains precedence.	Peak Refractory Rule
A5	Decimal Rounding	All floating-point results round to 1 decimal place using half-up arithmetic (`round(v, 1)`).	Decimal Rounding Rule
A6	Result Mode Output	Both offline and online results are saved and printed; the `mode` field disambiguates them.	Mode Flagging Rule
A7	Window Grace Period	Online processor waits at most 3.0 seconds after window end before emitting partial window results.	Window Grace Period
A8	Power-Cut Stated Bound	Worst-case uncommitted sample loss on power cut is 400 samples/sensor (1 second batch commit).	Power-Cut Recovery Bound
A9	Session Scope	Sessions are uniquely identified and partitioned by the mock's 32-bit hexadecimal `session_id`.	Session Scope Definition

7. AI TOOL USAGE DISCLOSURE

Tools Used: Antigravity IDE (Google DeepMind) with Gemini 3.8 Flash & Claude 3.7 Sonnet (via interactive agent pair-programming).

Used For: Code drafting across pipeline modules (reader, store, uploader, processing, cloud API), unit test generation, failure injection test scripts, HTML/PDF dossier styling, and initial design note drafting.

Not Used For: Core architectural selection (SQLite WAL, index-based monotone counters, and explicit loss accounting were determined by the engineer), physical hardware verification, or evaluating final pass/fail judgements.

Human Review Performed:

- Audited `processing.py` mathematical formulas line-by-line (peak detection, 100-sample gap, 5-second window HR formula).
- Executed and verified all 38 pytest unit tests locally (100% pass).
- Executed end-to-end failure injection tests (network cut, process kill, disk full, corrupt reply) and validated raw JSONL diff outputs.
- Audited 1-hour resource CSV logs and evaluated resource metrics against target Pi 4 limits.

Date: October 2026

8. FRESH-BOARD INSTALLATION & OPERATIONAL GUIDE

To deploy the submission on a clean Raspberry Pi 4 running Raspberry Pi OS 64-bit (Debian Bookworm):

```
# 1. Clone repository
git clone https://github.com/adityas777/Maya-Cardio.git /opt/pipeline-src
cd /opt/pipeline-src

# 2. Execute automated setup script (installs dependencies, venv, systemd units)
sudo bash setup_pi.sh

# 3. Configure credentials
sudo bash -c 'echo "PIPELINE_CLOUD_TOKEN=your_bearer_token" > /etc/pipeline.env'
sudo chmod 600 /etc/pipeline.env

# 4. Start systemd services
sudo systemctl start biosignal-mock
sudo systemctl start pipeline

# 5. Inspect status and verify live collection
pipeline status
```

Required CLI Commands:

Command	Operational Function & Output Format
`pipeline status`	Prints 1 JSON line: current mode (online/offline), samples saved, samples in cloud, and backlog count.
`pipeline export --session <ID> --from device`	Streams JSON lines of saved samples and loss records directly from local SQLite database.
`pipeline export --session <ID> --from cloud`	Streams JSON lines of samples and loss records fetched from the Cloud API.
`pipeline results --session <ID>`	Outputs 1 JSON line per window with calculated statistics, channel min/max/mean, magnitude, and heart rate.

9. VERIFICATION & SUBMISSION FILE MANIFEST

The submission package contains the following structure:

```
submission/
├── MayaCardio_Submission_Dossier.pdf  <-- THIS MASTER DOCUMENT (with video link & evidence)
├── DESIGN_NOTE.pdf                    <-- Strict 2-page design document
├── README.md                          <-- Installation, assumptions, and command reference
├── AI_DISCLOSURE.md                   <-- AI tool usage disclosure document
```

```
├── resources_report.md              <-- 1-hour online mode figures & test environment report
├── evidence/                        <-- Full measured CSVs, test logs, and export diffs
│   ├── resources_1h.csv             <-- 3,603s continuous resource time-series
│   ├── ramp_export.jsonl            <-- Deterministic ramp export dataset
│   ├── 01_process_kill/             <-- Process kill run log & status before/after
│   ├── 02_power_cut/                <-- Software power cut simulation summary & bounds check
│   ├── 03_network_cut/              <-- Network cut log (1s switch, 40s catchup) & cloud diff
│   ├── 04_mock_restart/             <-- Mock session restart logs & session export
│   ├── 05_disk_full/                <-- Disk full survival log & recovery validation
│   ├── 06_clock_change/             <-- Clock jump logs & monotonic stability proof
│   ├── 07_corrupt_and_duplicate/    <-- Fault proxy CRC rejection & deduplication log
│   └── 08_sensor_silent/            <-- Sensor silent partial window & missing count log
└── pipeline/                        <-- Clean source code repository (src, tests, scripts, systemd)
```